

Donner un disque à un agent

Rôles, confinement et refus, sur le cas d'un serveur MCP filesystem.

Denis Lamard

Concepteur et intégrateur d'agents IA

github.com/denislamard

Version 1.0, 2026

Sommaire

1. Le serveur que tout le monde branche en premier
2. Ce que « donner accès à un dossier » veut dire
3. Le rôle est un plafond
4. Le point de passage unique
5. Les trois contrôles, dans cet ordre
6. Les motifs de refus
7. Ce que le filtrage par nom ne fait pas
8. La surface d'outils suit la configuration
9. Effacer sans effacer
10. Tout valider au chargement
11. Parcourir sans traverser le bruit
12. Le journal d'audit
13. Ce que ce modèle laisse passer
14. Ce que je retiens
15. À propos

Annexe A. Lexique

Annexe B. Revue avant d'ouvrir un dossier à un agent

Annexe C. La configuration, champ par champ

Annexe D. Les refus et leurs messages

1. Le serveur que tout le monde branche en premier

« Tu peux regarder dans mon dossier de code ? »

La demande est banale et la réponse tient en trois lignes de configuration. On déclare un serveur MCP filesystem, on lui passe un chemin, l'assistant lit les fichiers. C'est souvent le premier serveur qu'on branche, le jour où on découvre le protocole, et c'est celui qui reste branché ensuite.

Le serveur de référence de l'écosystème, @modelcontextprotocol/server-fileSystem, prend ses répertoires en arguments de ligne de commande. Sur chacun d'eux il expose la lecture, l'écriture et la suppression. Un seul niveau d'accès pour tous les chemins déclarés, et aucun filtrage sur ce qu'ils contiennent : un .env posé à la racine d'un dépôt se lit comme n'importe quel autre fichier texte.

Ma configuration compte 5 racines, et c'est elle qui sert de fil rouge d'un bout à l'autre de ce document.

Racine	Ce qu'elle contient	Ce que je veux qu'un agent y fasse
loom	le socle d'orchestration que je développe	lire, modifier
loom-docs	la documentation de ce socle	lire
publication	les guides PDF et la page qui les présente	lire, modifier
poc	des prototypes en cours	lire, modifier, ranger
claude	un dossier de travail jetable	tout

Trois besoins distincts apparaissent dès la deuxième ligne. La documentation doit être citée et jamais réécrite. Un dépôt de code accepte qu'on modifie un fichier, parce que git montre la modification et sait la défaire ; la suppression d'un fichier non suivi, elle, ne se rattrape nulle part. Un dossier de travail assume les trois opérations, puisque son contenu est jetable par construction.

loom-fs est le serveur que j'ai écrit pour tenir cette distinction. Ce cahier décrit son modèle de sécurité : ce qu'il garantit, par quel mécanisme, et où la garantie s'arrête. Le code tient en 3 fichiers Python et 47 tests, dont 24 portent sur la résolution de chemin et la validation de la configuration.

Ce cahier est une étude de cas. Il décrit une implémentation précise plutôt qu'une méthode générale. Les quatre décisions qu'il isole se transposent néanmoins à tout serveur MCP qui touche des ressources réelles : le plafond par rôle, le point de passage unique, la liste de refus non désactivable, et la validation au chargement.

Ce que le protocole ne décide pas

MCP normalise la façon dont un serveur déclare ses outils et dont un client les appelle. Il ne dit rien de ce qu'un outil a le droit de faire. Un serveur MCP est un programme ordinaire, lancé sur la machine de l'utilisateur, avec les droits de sa session. Le protocole transporte l'appel et laisse la politique d'accès entièrement à la charge de celui qui écrit le serveur.

La répartition est saine, et elle a une conséquence pratique. Quand vous branchez un serveur écrit par quelqu'un d'autre, vous héritez de sa politique d'accès sans qu'elle figure nulle part dans le protocole. Pour la connaître, il faut aller la lire dans son code.

2. Ce que « donner accès à un dossier » veut dire

La phrase recouvre 3 questions que l'on traite souvent comme une seule.

- **Le périmètre.** Quels chemins existent, du point de vue de l'agent. Ce qui est en dehors n'a pas de nom qu'il puisse prononcer.
- **La permission.** Quelles opérations sont possibles à l'intérieur de ce périmètre, sachant qu'elles n'ont aucune raison d'être les mêmes d'un répertoire à l'autre.
- **Le filtrage.** Quels fichiers, bien qu'à l'intérieur du périmètre et couverts par la permission, restent inaccessibles et invisibles.

Le serveur standard répond à la première question. Les deux autres sont alors reportées sur l'utilisateur, qui doit choisir des répertoires assez étroits pour qu'un niveau d'accès unique et l'absence de filtrage ne posent pas de problème. En pratique, on déclare son dossier de projets entier et on n'y pense plus.

Une chaîne fournie par un modèle est une entrée hostile

Le chemin qui arrive dans un outil est produit par un modèle de langage, à partir d'une conversation qui contient, entre autres choses, le contenu des fichiers déjà lus. Personne ne l'a tapé. Son origine est un texte que l'agent a rencontré quelque part, et ce texte peut avoir été écrit par un tiers.

Le traitement qui en découle est celui qu'on applique à une entrée réseau. On ne suppose rien de sa forme. On ne la concatène pas à un chemin de base en considérant l'affaire close. La vérification porte sur le chemin résolu, parce que les segments ../ et les liens symboliques ne se lisent pas dans la chaîne d'origine.

Le reste de ce cahier décrit les quatre mécanismes qui appliquent ce principe : le rôle qui plafonne les droits d'une racine, la fonction unique qui résout et autorise, la liste de refus qui rend certains noms inatteignables, et la validation qui refuse de démarrer sur une configuration ambiguë.

3. Le rôle est un plafond

Chaque racine déclare un rôle. Le rôle fixe le maximum de droits accordables sur ce répertoire.

Rôle	Plafond	Ce que je mets dedans
reference	read	documentation, spécifications, données de référence
sourcecode	read, write	un dépôt de code que l'agent modifie
workspace	read, write, delete	un dossier de travail jetable

Une racine peut descendre sous son plafond avec le champ permissions. Elle ne peut pas monter au-dessus. Un workspace ramené à la lecture seule le temps d'une intervention est une configuration valide ; un sourcecode auquel on ajoute delete fait échouer le chargement.

```
ceiling = ROLE_CEILING[self.role]
granted = ceiling if self.permissions is None else self.permissions
if extra := granted - ceiling:
    names = ", ".join(sorted(extra))
    raise ValueError(
        f"root {self.name!r}: role {self.role} does not allow {names}; "
        f"use a wider role or drop the permission"
    )
if not granted:
    raise ValueError(f"root {self.name!r}: no permission granted")
```

Le message est écrit pour la personne qui vient d'éditer le fichier. Il nomme la racine, le rôle, le droit refusé, et les deux façons de s'en sortir. Un message d'erreur de configuration est lu une fois, dans un moment d'agacement, et il a intérêt à contenir la réponse.

Pourquoi un rôle plutôt qu'une liste de droits

Une liste de permissions décrit la mécanique. Un rôle décrit l'intention. Six mois après avoir écrit sa configuration, on relit `role = "reference"` et on sait ce qu'est ce répertoire ; on relit `permissions = ["read"]` et on sait seulement ce qu'on avait tapé ce jour-là.

Le rôle porte aussi un défaut qui n'a rien à voir avec les permissions : la taille maximale d'un fichier lisible, 1 Mio pour une racine reference, 2 Mio pour du code, 8 Mio pour un dossier de travail. Le raisonnement est que les gros fichiers utiles arrivent dans le dossier de travail, alors qu'un fichier d'un mégaoctet dans un dépôt de code est presque toujours quelque chose que l'agent n'a pas besoin de lire en entier. Le champ `max_read_bytes` remplace ce défaut quand le cas le demande.

4. Le point de passage unique

Une fonction transforme les chaînes fournies par le modèle en chemins réels : `security.resolve`. Tous les outils passent par elle, et aucun ne construit de chemin par lui-même.

```
def _resolve(root_name: str, path: str, operation: Permission) -> Resolved:
    try:
        root = config.root(root_name)
    except KeyError as exc:
        raise ToolError(str(exc.args[0])) from None
    try:
        return resolve(root, path, operation)
    except AccessDenied as exc:
        raise ToolError(str(exc)) from None

@mcp.tool
async def read_file(root: str, path: str) -> str:
    """Read a UTF-8 text file. Binary and oversized files are refused."""
    target = _resolve(root, path, Permission.READ)
    ...
```

L'intérêt de cette forme est qu'elle laisse un seul endroit à vérifier. Un deuxième site de construction de chemin serait une deuxième occasion de se tromper, et il se serait trompé autrement, donc les tests écrits pour le premier ne l'auraient pas attrapé. Les 8 outils qui touchent au disque tiennent chacun en une quinzaine de lignes, parce que la partie difficile a déjà eu lieu à la première ligne du corps.

`resolve` rend un objet `Resolved` qui porte 3 champs : la racine, le chemin réel, et le chemin relatif à cette racine. Les outils travaillent sur le chemin réel et affichent le chemin relatif. Le modèle ne voit de chemin absolu que pour les racines elles-mêmes, rendues par `list_roots` ; à l'intérieur, tout s'exprime relativement.

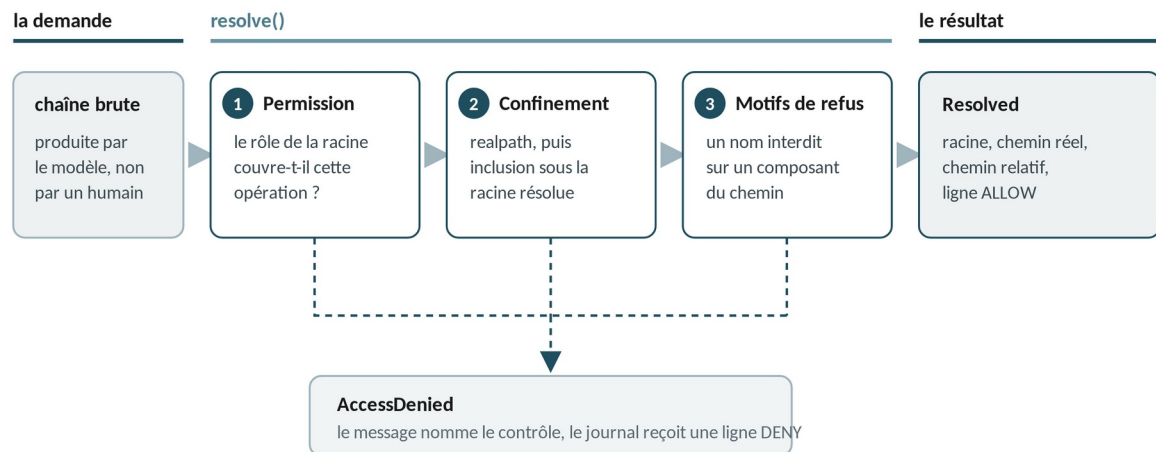
Le refus est tracé au même endroit

`resolve` écrit une ligne de journal dans les deux cas de figure. Un `ALLOW` pour chaque résolution acceptée, un `DENY` pour chaque refus, avec la racine, le chemin brut demandé et la raison. Le refus est construit par une petite fonction dédiée, ce qui a un effet simple : aucune branche ne peut refuser sans laisser de trace, puisque la trace est dans le constructeur de l'exception.

```
def _deny(reason: str, *, root: str, raw: str) -> AccessDenied:
    audit.warning("DENY root=%s path=%r reason=%s", root, raw, reason)
    return AccessDenied(reason)
```

5. Les trois contrôles, dans cet ordre

`resolve` applique 3 contrôles. L'ordre est fixe, et le message d'erreur nomme toujours celui qui a mordu.



1. Permission. L'opération demandée est-elle dans les droits de la racine ? Le refus s'écrit root 'loom-docs' (role reference) grants [read], not write.

2. Confinement. Le chemin résolu tombe-t-il sous la racine résolue ? Le refus s'écrit path escapes the root.

3. Motifs de refus. Un composant du chemin relatif correspond-il à un motif interdit ? Le refus s'écrit matches deny pattern '.env'.

Pourquoi la permission passe en premier

C'est le contrôle le moins cher, et surtout le seul qui ne dépende pas de l'état du disque. Refuser une écriture sur une racine en lecture seule doit donner le même résultat que le fichier existe ou non. Placer ce contrôle après la résolution ferait varier le message d'erreur selon l'arborescence, et un message qui varie selon l'arborescence renseigne l'appelant sur l'arborescence.

Le confinement porte sur les chemins résolus

`os.path.realpath` est appliqué à la cible, et la racine a été résolue une fois pour toutes au chargement de la configuration. L'inclusion est vérifiée sur ces deux résultats.

```

candidate = Path(raw)
target = candidate if candidate.is_absolute() else root.real / candidate
real = Path(os.path.realpath(target))

if real != root.real and root.real not in real.parents:
    raise _deny("path escapes the root", root=root.name, raw=raw)
  
```

Résoudre avant de comparer est ce qui fait échouer un lien symbolique sortant ici, dans la fonction d'autorisation, au lieu du `open()` qui aurait suivi. Le test qui couvre ce cas crée un lien `inside/escape` vers un répertoire voisin, puis vérifie que la lecture de `escape/secrets.txt` est refusée avec le message de confinement.

L'effet de bord est agréable : une racine atteinte à travers un lien symbolique fonctionne normalement, puisqu'elle a été résolue au chargement. C'est le cas courant sur une machine où le dossier de projets est un lien vers un autre disque.

Les cibles inexistantes se résolvent correctement. `realpath` résout le préfixe qui existe et concatène le reste, donc la création d'un fichier dans un répertoire encore à créer ne bute pas sur le contrôle de confinement. Sans cette propriété, il aurait fallu un chemin de code séparé pour l'écriture, et on retombait sur le problème des deux endroits à vérifier.

Les motifs s'appliquent à chaque composant

Le troisième contrôle ne regarde pas le nom de base du fichier. Il parcourt tous les composants du chemin relatif.

```
for pattern in root.deny_globs:
    if any(fnmatch.fnmatch(part, pattern) for part in rel.parts):
        raise _deny(f"matches deny pattern {pattern!r}", root=root.name, raw=raw)
```

`pkg/.venv/lib/thing.py` est donc refusé par le motif `.venv`, qui ne correspond pourtant à aucun nom de fichier de ce chemin. Un contrôle limité au nom de base laisserait passer l'intégralité du contenu d'un répertoire interdit dès lors qu'on en demande un fichier directement.

Un octet nul dans la chaîne est refusé avant tout le reste. Il tronque les chemins au niveau des appels système sous-jacents, et une chaîne vérifiée avant troncature ne dit rien de la chaîne effectivement utilisée après.

6. Les motifs de refus

Trois niveaux s'additionnent, et aucun ne peut retrancher : la liste de base inscrite dans le code, la liste `deny` de la configuration qui s'applique à toutes les racines, et la liste `deny` d'une racine particulière. La propriété `deny_globs` d'une racine rend simplement leur concaténation.

```
BASE_DENY: tuple[str, ...] = (
    ".env", ".env.*",
    "*.pem", "*.key", "*.p12", "*.pfx",
    "id_rsa*", "id_ecdsa*", "id_ed25519*",
    ".ssh", ".aws", ".gnupg", ".netrc", ".npmrc", ".pyirc",
    ".git", ".venv", "node_modules", "__pycache__", TRASH_DIR,
)
```

Vingt motifs, répartis en quatre familles : les fichiers d'environnement, les clés privées et certificats, les répertoires de configuration personnelle où dorment des identifiants, et les répertoires de machine qui gonflent un dépôt sans rien apprendre à personne.

Pourquoi la liste de base reste dans le code

Si elle passait en configuration, elle deviendrait retirable. C'est exactement le genre de ligne qu'on commente un soir, pour un test, en se disant qu'on la remettra demain. Une garantie qu'un fichier de configuration peut désactiver n'est plus une garantie.

La tentative est fermée explicitement. Le modèle Config porte extra = "forbid", donc un base_deny = [] posé dans le TOML fait échouer le chargement au lieu d'être ignoré en silence. Un test vérifie ce comportement, parce qu'un champ ignoré silencieusement donne exactement la même impression de fonctionnement qu'un champ pris en compte.

Un fichier refusé n'apparaît nulle part

Les motifs filtrent aussi les résultats. list_directory saute les entrées interdites, et le parcours qui alimente search_files ne descend pas dans les répertoires interdits. Le modèle ne sait pas que le fichier existe.

Ce point compte plus qu'il n'en a l'air, parce qu'un refus est une information. Un agent qui reçoit matches deny pattern '.env' vient d'apprendre qu'il y a un .env à cet endroit, et un agent détourné par une injection sait désormais où insister. Une entrée absente d'un listing ne produit aucun message.

7. Ce que le filtrage par nom ne fait pas

Le filtrage porte sur des noms. Une clé d'API codée en dur dans settings.py se lit sans obstacle. Un jeton collé dans un README se lit aussi. Le mécanisme attrape des conventions, et une convention est une heuristique.

J'ai regardé l'option d'un filtrage de contenu, sous la forme d'expressions régulières appliquées à chaque lecture sur les formes connues de secrets. Elle coûte un balayage complet de chaque fichier lu. Elle produit des faux positifs sur du code de test, où les clés factices ressemblent à des vraies. Et son mode de défaillance est pire que le problème qu'elle traite : un fichier rendu à moitié caviardé fait raisonner l'agent sur du code mutilé, sans qu'il sache qu'il lui manque quelque chose.

La frontière réelle est le choix des racines. Le reste est de la défense en profondeur, utile contre l'accident et contre l'agent distrait, inopérante contre quelqu'un qui a déjà la main sur le contenu des fichiers.

8. La surface d'outils suit la configuration

Les outils d'écriture ne sont enregistrés que si au moins une racine accorde write. Ceux de suppression seulement si une racine accorde delete. Le catalogue est construit au démarrage, à partir de la configuration chargée.

```
if any(root.grants(Permission.WRITE) for root in config.roots):  
    @mcp.tool
```

```

async def write_file(root: str, path: str, content: str,
                    overwrite: bool = False) -> str:
    ...

```

Configuration	Ce que le modèle voit	Nombre
racines reference seulement	list_roots, list_directory, read_file, search_files	4
au moins une racine en write	les 4 précédents, plus write_file, create_directory, edit_file, move_file	8
au moins une racine en delete	les 8 précédents, plus delete	9

La différence avec un refus à l'appel se compte en tours de conversation. Un outil annoncé est une invitation : le modèle le choisit, dépense un appel, reçoit un refus, et repart souvent sur une variante du même chemin en supposant que le problème venait de l'argument. Un catalogue qui décrit exactement ce qui est possible supprime cette boucle, et il coûte moins de jetons à transporter puisqu'il est réexpédié à chaque appel au modèle.

La contrepartie est qu'un changement de périmètre demande de redémarrer le serveur, donc de fermer l'application cliente. Je l'accepte. Un serveur qui recalculerait sa surface d'outils en cours de session ferait changer le catalogue sous le modèle, au milieu d'une conversation où il a déjà planifié plusieurs appels.

9. Effacer sans effacer

delete ne supprime rien. Il déplace la cible vers une corbeille interne à la racine, horodatée en UTC.

```

if target.real == target.root.real:
    raise ToolError("refusing to delete the root itself")
if not target.real.exists():
    raise ToolError(f"{target} does not exist")
stamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S%f")
trash = target.root.real / TRASH_DIR / stamp
trash.mkdir(parents=True, exist_ok=True)
target.real.rename(trash / target.real.name)

```

- La corbeille appartient à la racine, donc le déplacement reste sur le même système de fichiers et ne recopie pas les données.
- Le répertoire .mcp-trash figure dans la liste de refus de base, donc l'agent ne peut ni lire la corbeille ni la vider.
- La racine elle-même est refusée comme cible, ce qui évite qu'un chemin « . » parfaitement bien formé emporte l'ensemble.

Le coût est réel. La corbeille grossit, la restauration est manuelle, et c'est moi qui la vide. Je l'accepte parce que l'opération inverse serait une suppression irréversible décidée par un modèle de langage à partir d'un nom de fichier qu'il a lu quelque part.

L'outil n'existe que si une racine porte le rôle workspace. Sur ma configuration, une racine sur cinq est concernée, et c'est un dossier dont je sais qu'il ne contient rien que je n'aie déjà ailleurs.

10. Tout valider au chargement

Le serveur ne démarre pas en mode dégradé. Toute incohérence lève à la lecture du fichier de configuration.

- Un répertoire déclaré qui n'existe pas.
- Deux racines qui portent le même nom.
- Deux racines dont l'une contient l'autre.
- Une racine à laquelle aucune permission n'est accordée.

Les racines imbriquées

Le cas mérite son test. Si `/dev/projet` est déclaré en sourcecode et `/dev/projet/scratch` en workspace, quelle permission s'applique à `/dev/projet/scratch/note.txt` ? Les deux réponses se défendent, et celle qui sortirait dépendrait de l'ordre de déclaration dans le fichier. Un comportement correct qui dépend de l'ordre des lignes d'un TOML est un piège posé pour plus tard.

```
for other_name, other in seen.items():
    if root.real == other or other in root.real.parents \
       or root.real in other.parents:
        raise ValueError(
            f"roots {other_name!r} and {root.name!r} overlap on disk; "
            f"nested roots make the effective permission ambiguous"
        )
```

Le contrôle porte sur les chemins résolus, donc deux racines qui se recouvrent à travers un lien symbolique sont attrapées elles aussi.

Pourquoi refuser de démarrer

Un serveur qui démarre avec 4 racines sur 5 a l'air de fonctionner. L'agent rapporte ensuite que le dossier n'existe pas, et on cherche la panne au mauvais endroit, parfois longtemps, parce que le client MCP ne montre pas la sortie d'erreur du serveur qu'il a lancé.

Le même raisonnement vaut pour `extra = "forbid"`. Un champ mal orthographié dans le TOML fait échouer le chargement. Sans cela, `max_search_result` écrit au singulier serait accepté, ignoré, et le plafond resterait à sa valeur par défaut sans que rien ne le signale.

C'est aussi la raison pour laquelle je lance le serveur en direct avant de le brancher dans le client. Il liste ses racines, leur rôle et leur chemin résolu, puis annonce le transport stdio. Trente secondes qui évitent de déboguer à travers une application.

11. Parcourir sans traverser le bruit

Le parcours de fichiers n'est pas un contrôle de sécurité. Il porte deux propriétés dont dépend la justesse du raisonnement de l'agent, ce qui suffit à le placer dans ce document.

Élaguer avant de descendre

`iter_files` confronte chaque entrée de répertoire aux motifs de refus avant de décider d'y descendre. Un `.venv` ou un `node_modules` sous la racine coûte un appel à `fnmatch` sur son nom.

```
for entry in entries:
    if is_hidden_by_policy(root, entry.name) or entry.is_symlink():
        continue
    rel = f"{prefix}/{entry.name}"
    if entry.is_dir(follow_symlinks=False):
        queue.append((entry.path, f"{rel}/"))
    elif entry.is_file(follow_symlinks=False):
        yield entry, rel
```

Mesuré sur un arbre de 24 300 fichiers dont 300 utiles, le reste étant du `.venv` et du `node_modules` : 1 224 ms pour l'implémentation précédente, 3,2 ms pour celle-ci, soit un facteur 380. La version précédente appelait `Path.glob` puis filtrait les résultats, donc elle traversait le bruit à chaque appel.

Un ordre déterministe

Le parcours est en largeur, avec un tri par répertoire. Deux appels identiques rendent la même liste dans le même ordre, et un résultat tronqué contient les correspondances les plus proches de la racine plutôt qu'une tranche arbitraire du disque.

Les liens symboliques sont ignorés en bloc pendant le parcours. Ça règle d'un coup la vérification du confinement entrée par entrée et les cycles, qui feraient tourner la boucle indéfiniment. Un répertoire illisible est sauté et journalisé, jamais fatal au parcours.

La troncature s'annonce

Le résultat d'une recherche porte un champ `truncated` et un champ `note`, en plus des correspondances.

```
{
  "matches": [{ "path": "loom_ia/agent.py", "line": 483, "preview": "..."}],
  "truncated": false,
  "examined_files": 36,
  "scanned_bytes": 222742,
  "skipped_too_large": 0,
  "skipped_binary": 0,
  "note": null
}
```

Une liste tronquée qui se présente comme complète produit une conclusion fausse et confiante : « ce symbole n'existe nulle part », alors que l'agent a vu les 200 premiers fichiers sur 3 000. Le champ note dit lequel des deux plafonds a mordu, le nombre de correspondances ou le budget d'octets, ce qui indique au passage comment resserrer la recherche.

Un écart assumé avec glob

Un motif sans séparateur correspond au nom de base à n'importe quelle profondeur. *.py trouve donc tous les fichiers Python de la racine. Le glob standard rendrait les fichiers Python du premier niveau, souvent aucun, et un agent en conclurait qu'il n'y a pas de Python dans ce dépôt. Pour la sémantique stricte, on écrit pkg/*.py.

Les motifs compilés sont mémorisés sur 512 entrées. Un motif vide, absolu, contenant .. ou portant un crochet non fermé est rejeté avec un message qui le cite.

Le filtre sur le contenu

L'argument contains déclenche une recherche en octets, sans décodage. Trois gardes s'appliquent avant chaque lecture : la taille du fichier est comparée au max_read_bytes de la racine, les 4 premiers Kio servent à détecter le binaire, et un budget global de 64 Mio par appel arrête le balayage. Chaque correspondance rend le chemin, le numéro de ligne et un extrait, ce qui évite un read_file de suivi et donc un aller-retour complet avec le modèle.

12. Le journal d'audit

Une ligne par résolution, acceptée ou refusée.

```
2026-09-11 14:22:07,481 loom_fs_mcp.audit INFO ALLOW root=loom path=loom_ia/agent.py op=read
2026-09-11 14:22:09,003 loom_fs_mcp.audit WARNING DENY root=loom path='./.env' reason=path escapes the root
2026-09-11 14:22:31,772 loom_fs_mcp.audit INFO ALLOW root=claude path=note.md op=write
```

Le journal ne contient aucun contenu de fichier. Des racines, des chemins relatifs, des opérations et des raisons de refus. Un journal qui porterait les contenus serait une seconde copie du disque, non filtrée celle-là, posée dans un fichier que personne ne surveille.

Purger sans casser le journal

La rotation est bornée par audit_log_max_bytes et audit_log_backups, 8 Mio et 3 fichiers de conservation par défaut, soit 32 Mio au total. Pour purger à chaud, il faut tronquer.

```
truncate -s 0 ~/.local/state/loom-fs-mcp/audit.log
```

Un rm laisse le serveur écrire dans un inode supprimé : les lignes continuent de partir quelque part, l'espace n'est pas rendu, et rien ne réapparaît avant l'arrêt du processus. Si vous passez par logrotate, la même raison impose copytruncate.

Dernier point, valable pour tout serveur stdio qui journalise dans un fichier : la rotation a lieu dans le processus qui écrit. Deux instances du serveur pointées sur le même fichier se marcheraient dessus au moment de renommer. Un fichier par instance, ou un WatchedFileHandler avec logrotate à l'extérieur.

13. Ce que ce modèle laisse passer

Cette partie existe parce qu'un lecteur qui découvre ces limites après coup a raison de se demander ce qu'on lui a caché d'autre.

Aucun quota, aucune limitation de débit

Un agent parti en boucle peut lire des milliers de fichiers. Avant d'ajouter un plafond, il faut voir ce qu'il ne ferait pas. Un quota n'est pas un contrôle de sécurité : si une injection détourne l'agent, un plafond à 200 fichiers laisse fuir 200 fichiers. Ce qu'un quota borne, c'est le coût et le rayon d'un emballement.

La notion de session n'existe pas non plus côté serveur. En stdio, le processus survit à toutes les conversations du client. Un compteur cumulatif finirait par se déclencher et bloquerait jusqu'au redémarrage de l'application, sans explication lisible pour la personne qui l'utilise. La forme qui tiendrait serait un seau à jetons régénéré dans le temps, comptant des octets plutôt que des appels : 10 000 petits fichiers ne coûtent rien, 50 fichiers de 8 Mio pèsent 400 Mio.

Le sujet deviendra pertinent le jour où ce serveur alimentera un agent que personne ne regarde. En usage interactif, la personne qui suit la conversation est le plafond.

Le refus de .git emporte l'historique

Pas de « pourquoi ce code est écrit comme ça », pas de « qu'est-ce qui a changé dans ce commit ». C'est la limite que je rencontre le plus souvent en pratique.

Ouvrir .git en accès fichier ne rendrait presque rien. Les objets sont compressés, donc le contrôle binaire les refuse. Ce qui resterait lisible, c'est .git/config, qui contient des identifiants quand une remote est en HTTPS, et les hooks, qui sont du code exécuté à la prochaine opération git.

Un outil git dédié en lecture seule serait la bonne réponse, et il coûte plus cher qu'il n'en a l'air.

- Il travaille au niveau du dépôt, donc sur un périmètre plus large que les racines déclarées.
- L'historique contourne la liste de refus, puisqu'un .env commité il y a deux ans est toujours dans les objets. Il faudrait dériver des pathspecs :(exclude,glob) de la même liste, et maintenir une seconde source de vérité.

- Lancer git dans un dépôt exécute la configuration de ce dépôt. Les alias, core.pager, diff.external et les filtres textconv sont autant de commandes shell. Sur un dépôt cloné, c'est une exécution de code arbitraire déclenchée par un outil annoncé en lecture seule.

Écarté pour l'instant. L'archéologie git se fait en session de développement, dans le dépôt, où git fonctionne nativement et où je vois ce que je lance.

Le processus tourne avec vos droits

Le serveur n'est isolé de rien. C'est un processus Python lancé par l'application cliente, avec les droits de la session, sans machine virtuelle ni utilisateur dédié. Tout ce qui précède est de la discipline interne à ce processus, et un défaut dans resolve donnerait accès au disque entier.

C'est la raison pour laquelle le choix des racines est la décision de sécurité, et pourquoi je garde cette fonction courte, avec un seul chemin de retour et sa propre batterie de tests.

L'injection indirecte n'est pas traitée

Un fichier lu par l'agent peut contenir des instructions destinées à l'agent. loom-fs ne détecte pas ce cas et ne prétend pas le faire. Ce qu'il fait, c'est borner ce qu'un agent détourné peut atteindre : les racines déclarées, les opérations de leur rôle, et rien qui corresponde à un motif de refus. La borne ne dépend pas de la lucidité du modèle, et c'est toute sa valeur.

14. Ce que je retiens

Le mécanisme qui fait le plus de travail est le plus ennuyeux à écrire. Une fonction de résolution de chemin, courte, appelée partout, testée en détail. Elle ne se démontre pas, elle n'apparaît dans aucune interface, et c'est elle qui décide.

La deuxième décision utile a été de faire échouer la configuration au chargement. Elle transforme des questions difficiles, du genre « quelle permission s'applique à ce fichier », en questions que le fichier de configuration n'a pas le droit de poser.

La troisième a été d'écrire noir sur blanc ce que le modèle ne couvre pas, dans le README du dépôt et dans la partie 13 de ce cahier. Un serveur de fichiers pour agent donne une impression de sûreté que son code ne soutient qu'en partie, et cette impression est elle-même un risque.

Reste la question que ce cahier ne tranche pas : jusqu'où monter en garantie avant de changer de catégorie. Un serveur qui tournerait dans une microVM, avec un système de fichiers monté depuis l'extérieur et un canal vsock pour seul lien, offrirait une frontière là où il n'y a aujourd'hui qu'une discipline. C'est un autre travail, et je le mène par ailleurs sur l'exécution de code produit par un agent.

15. À propos

Je conçois et j'intègre des agents IA destinés à tourner en production, pour des PME et des artisans. Ce cahier décrit un outil de mon propre poste de travail, celui par lequel passent les agents avec lesquels j'écris le reste.

J'écris et je maintiens un socle d'orchestration public. Il porte les mécanismes décrits dans cette collection : validation des arguments avant appel, exécution parallèle bornée, checkpoints et reprise.

Le code est public : github.com/denislamard/loom

Le détail technique et les cas d'usage : denislamard.github.io

Annexe A. Lexique

Terme	Ce qu'il désigne ici
racine	un répertoire déclaré dans la configuration, seul endroit du disque qu'un outil peut atteindre
rôle	l'étiquette d'une racine, qui fixe le maximum de droits accordables dessus
plafond	l'ensemble des permissions qu'un rôle autorise ; une racine peut descendre en dessous
confinement	la vérification que le chemin résolu tombe bien sous la racine résolue
motif de refus	un motif de nom qui rend un fichier inatteignable et invisible, à tous les niveaux du chemin
realpath	la résolution d'un chemin en chemin absolu sans lien symbolique ni segment relatif
corbeille	le répertoire .mcp-trash d'une racine, où delete déplace ce qu'on lui demande d'effacer
stdio	le transport MCP par entrée et sortie standard, où le serveur est un processus fils du client
glob	un motif de nom de fichier, avec * et ? dans un segment et à travers les répertoires

Annexe B. Revue avant d'ouvrir un dossier à un agent

Une page à parcourir avant de déclarer une racine sur des données réelles. Elle vaut pour loom-fs comme pour n'importe quel serveur MCP qui touche un système de fichiers.

Périmètre

- Chaque racine est le répertoire le plus étroit qui permette encore de travailler.
- Aucune racine ne contient une autre racine, y compris à travers un lien symbolique.
- Le répertoire personnel, la racine du disque et les répertoires système ne sont déclarés nulle part.
- Ce que contient chaque racine a été regardé une fois, en entier, avant de la déclarer.

Permissions

- Le rôle de chaque racine correspond à ce qu'on veut réellement y faire, pas à ce qui est commode.
- Les racines en écriture sont sous contrôle de version, ou sauvegardées.
- La suppression est accordée sur les seuls répertoires dont le contenu est reproductible.
- Une tentative d'élargir les droits au-delà du rôle fait échouer le démarrage, et ce comportement a été vérifié une fois.

Filtrage

- La liste de refus de base n'est modifiable par aucun fichier de configuration.
- Les motifs s'appliquent à tous les composants d'un chemin, et pas seulement au nom de fichier.
- Un fichier refusé n'apparaît ni dans un listing ni dans un résultat de recherche.
- Les extensions propres au projet qui portent des données sensibles ont été ajoutées à la liste deny de la configuration.

Exploitation

- Le serveur a été lancé en direct une fois, et il liste ses racines avant d'annoncer son transport.
- Le journal d'audit est écrit dans un fichier, sa rotation est bornée, et sa purge se fait par troncature.
- Les messages de refus nomment la règle qui a mordu.
- Le catalogue d'outils présenté au modèle correspond aux permissions réellement accordées.

Annexe C. La configuration, champ par champ

La configuration est un fichier TOML dont le chemin est passé au serveur par `--config`. Elle décrit l'arborescence réelle d'une machine, ce qui est une raison suffisante pour la garder hors du dépôt.

```

audit_log = "/home/denis/.local/state/loom-fs-mcp/audit.log"
audit_log_max_bytes = 8388608 # 8 Mio par fichier
audit_log_backups = 3 # audit.log.1 a .3, soit 32 Mio au total

max_search_results = 200
max_search_bytes = 67108864 # budget total scanne par recherche

deny = ["*.sqlite3", "*.dump"]

[[roots]]
name = "loom"
path = "/home/denis/dev/loom/src"
role = "sourcecode"

[[roots]]
name = "loom-docs"
path = "/home/denis/dev/docs"
role = "reference"

[[roots]]
name = "claude"
path = "/home/denis/temp"
role = "workspace"
deny = ["*.tmp"]

```

Les champs d'une racine

Champ	Obligatoire	Détail
name	oui	^[a-z0-9][a-z0-9._-]{0,31}\$, unique dans la configuration
path	oui	doit exister et être un répertoire ; ~ et variables d'environnement développés
role	oui	reference, sourcecode ou workspace
permissions	non	sous-ensemble du plafond du rôle ; par défaut, le plafond entier
deny	non	motifs supplémentaires, appliqués à cette racine seulement
max_read_bytes	non	remplace le défaut du rôle : 1, 2 ou 8 Mio

Les champs globaux

Champ	Défaut	Détail
deny	vide	motifs ajoutés à toutes les racines, en plus de la liste de base
audit_log	aucun	sans ce champ, les traces ne partent que sur la sortie d'erreur
audit_log_max_bytes	8 Mio	taille d'un fichier de journal avant rotation
audit_log_backups	3	nombre de fichiers conservés

Champ	Défaut	Détail
max_search_results	200	correspondances rendues au maximum par appel
max_search_bytes	64 Mio	octets balayés au maximum par appel, filtre contains

Annexe D. Les refus et leurs messages

Tous ces messages remontent au modèle tels quels, sous forme d'erreur d'outil. Ils sont écrits pour qu'il puisse corriger son appel au tour suivant sans supposer.

Situation	Message	Où le contrôle a lieu
écriture demandée sur une racine reference	root 'loom-docs' (role reference) grants [read], not write	resolve, contrôle 1
chemin qui sort de la racine, par .. ou par lien symbolique	path escapes the root	resolve, contrôle 2
nom interdit sur un composant du chemin	matches deny pattern '.env'	resolve, contrôle 3
octet nul dans le chemin	null byte in path	resolve, avant résolution
racine inconnue	unknown root 'x'; available: claudé, loom, loom-docs, poc, publication	config.root
fichier plus gros que le plafond de la racine	... is 3145728 bytes, over the 2097152 byte limit for this root	read_file
fichier binaire	... looks binary	read_file
écrasement sans overwrite	... exists; pass overwrite=true to replace it	write_file
remplacement non unique	`old` matches 3 times in ...; it must match exactly once	edit_file
suppression de la racine	refusing to delete the root itself	delete
motif de recherche malformé	invalid pattern './/*.py'	compile_glob

Le dernier point de cette liste est celui qui a le plus changé ma façon d'écrire des messages d'erreur. Un outil dont le refus nomme la règle et l'objet fautif se corrige au tour suivant. Un outil qui rend « erreur » fait recommencer l'agent à l'aveugle, et il recommence.